BASIC INTERACTIVE GRAPHIC USER INTERFACE (GUI)

INDIVIDUAL ASSIGNMENT

PROG209 EVENT DRIVEN PROGRAMMING
MAIMUNATU A. R. MASSAQUOI 905005569 | BIT 2101 SEMESTER 5

TABLE OF CONTENTS

INTRODUCTION 2

OBJECTIVES 2

SOFTWARE AND TOOLS USED 2

SYSTEM DESIGN / WIREFRAME 3

IMPLEMENTATION 4

- 5.1 JFrame 4
- 5.2 JPanel 4
- 5.3 JLabel 4
- 5.4 ImageIcon 4
- 5.5 Inheritance 4
- 5.6 Constructor 4

SOURCE CODE 5

APPLICATION SCREENSHOTS 9

TESTING 11

CONCLUSION 12

REFERENCE 13

INTRODUCTION

This project is a simple Graphical User Interface (GUI) application built with Java Swing. It makes use of core components such as JFrame, JPanel, JLabel, and ImageIcon, and applies object-oriented programming principles, notably inheritance and constructors. The application also includes two interactive features: users can change the text shown in the JLabel and update the background color of the JPanel.

OBJECTIVES

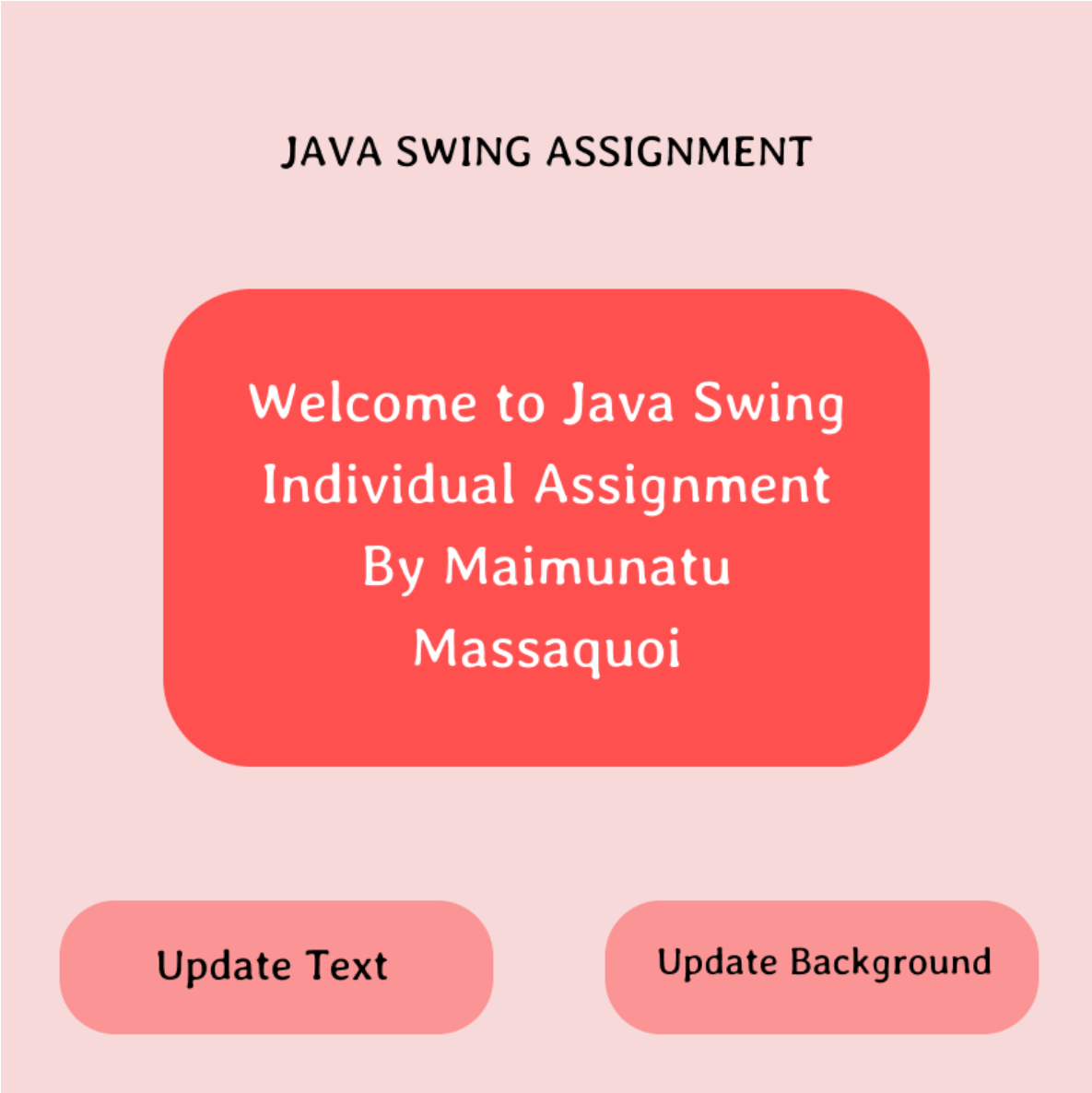
- Develop a simple GUI application using Java Swing.
- Set up a JFrame titled "Java Swing Assignment" with dimensions of 800 x 800 pixels.
- Build a JPanel and place it inside the JFrame.
- Display the required welcome message using a JLabel.
- Apply inheritance by having the main class extend JFrame.
- Use a constructor to initialise the application's components.
- Set an ImageIcon as the application's icon.
- Enable the JLabel text to be changed through user interaction.
- Enable the JPanel background colour to be changed through user interaction.
- Produce a wireframe in Figma and document how the design was implemented.

SOFTWARE AND TOOLS USED

- IntelliJ IDEA
- Java Swing
- Figma
- GitHub

SYSTEM DESIGN / WIREFRAME

The interface was designed in an 800 x 800 Figma frame. The wireframe includes a title area, a central welcome label, and two action buttons: one updates the JLabel text, while the other changes the JPanel background colour. The supplied wireframe-reference.png serves as a guide for building the final Figma wireframe.



IMPLEMENTATION

5.1 JFrame

The JFrame serves as the main window of the application. The program creates a JFrame variable named frame, sets its title to "Java Swing Assignment", and gives it dimensions of 800 x 800 pixels. The window is centred on the screen and configured to close when the user clicks the close button.

5.2 JPanel

The JPanel acts as the main content area of the application, holding the JLabel and the two buttons. When the Change Background button is clicked, the program updates the JPanel's background colour.

5.3 JLabel

The JLabel displays the required welcome message. When the Change Text button is clicked, the original message is replaced with a confirmation message, demonstrating dynamic interaction.

5.4 ImageIcon

An ImageIcon loads the custom application icon from the images folder. The icon is then applied to the JFrame using setIconImage().

5.5 Inheritance

Inheritance is demonstrated through the declaration public class SwingAssignment extends JFrame. This allows SwingAssignment to inherit the functionality of the JFrame class, fulfilling the inheritance requirement.

5.6 Constructor

The constructor public SwingAssignment() initialises the GUI components and configures the application. It runs automatically when new SwingAssignment() is called in the main method.

SOURCE CODE

```
import javax.swing.*;
import javax.swing.border.AbstractBorder;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class SwingAssignment extends JFrame {

    // Required JFrame, JPanel and JLabel variables
    private JFrame frame;
    private JPanel panel;
    private JLabel label;

    private JButton changeTextButton;
    private JButton changeBackgroundButton;

    // Constructor
    public SwingAssignment() {

        // Create and configure the JFrame
        frame = new JFrame("Java Swing Assignment");

        // Create the JPanel
        panel = new JPanel();

        // Create the JLabel with centered text alignment
        label = new JLabel(
                "Welcome to Java Swing Individual Assignment by Maimunatu A. R. Massaquoi",
                SwingConstants.CENTER
        );

        // Create buttons for the additional functionality
      changeTextButton = new JButton("Update Text");
        changeBackgroundButton = new JButton("Change Background");
```

```
// Configure the panel with GridBagLayout for perfect vertical/horizontal centering
panel.setLayout(new GridBagLayout());
panel.setBackground(Color.PINK);

// Improve the JLabel appearance
label.setFont(new Font("Tahoma", Font.BOLD, 16)); // Slightly reduced size to fit nicely in 800px width
label.setOpaque(true);
label.setBackground(Color.RED); // Background color inside the curved border
label.setForeground(Color.WHITE); // Text color

// Apply padding inside the label and add the custom Curved Border (20px radius, 2px thickness)
label.setBorder(BorderFactory.createCompoundBorder(
        new RoundedBorder(Color.LIGHT_GRAY, 2, 20),
        BorderFactory.createEmptyBorder(15, 20, 15, 20) // Padding inside the curve
));

// Layout constraints to stack components vertically and center them
GridBagConstraints gbc = new GridBagConstraints();
gbc.gridx = 0;
gbc.gridy = 0;
gbc.insets = new Insets(15, 10, 15, 10);
gbc.fill = GridBagConstraints.HORIZONTAL;
panel.add(label, gbc);

// Panel container for buttons to sit side-by-side cleanly
JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 20, 0));
buttonPanel.setOpaque(false); // Keeps background color seamless
buttonPanel.add(changeTextButton);
buttonPanel.add(changeBackgroundButton);

gbc.gridy = 1;
panel.add(buttonPanel, gbc);

// Add the JPanel to the JFrame
frame.add(panel);
```

```
    // Change the JLabel text when the button is clicked
    changeTextButton.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            label.setText("Mr. Amandus, The text has been updated successfully! Thank you.");
        }
    });

    // Change the JPanel background when the button is clicked
    changeBackgroundButton.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e) {
            panel.setBackground(Color.WHITE);
        }
    });

    // Add the custom application icon
    try {
        ImageIcon icon = new ImageIcon("images/java_icon.png");
        frame.setIconImage(icon.getImage());
    } catch (Exception e) {
        System.out.println("Icon missing; defaulting to standard system icon.");
    }

    // JFrame configuration
    frame.setSize(800, 800);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setLocationRelativeTo(null);
    frame.setResizable(false);
    frame.setVisible(true);
}

// Custom class to draw a beautiful anti-aliased curved border
private static class RoundedBorder extends AbstractBorder {
    private final Color color;
    private final int thickness;
    private final int radius;
```

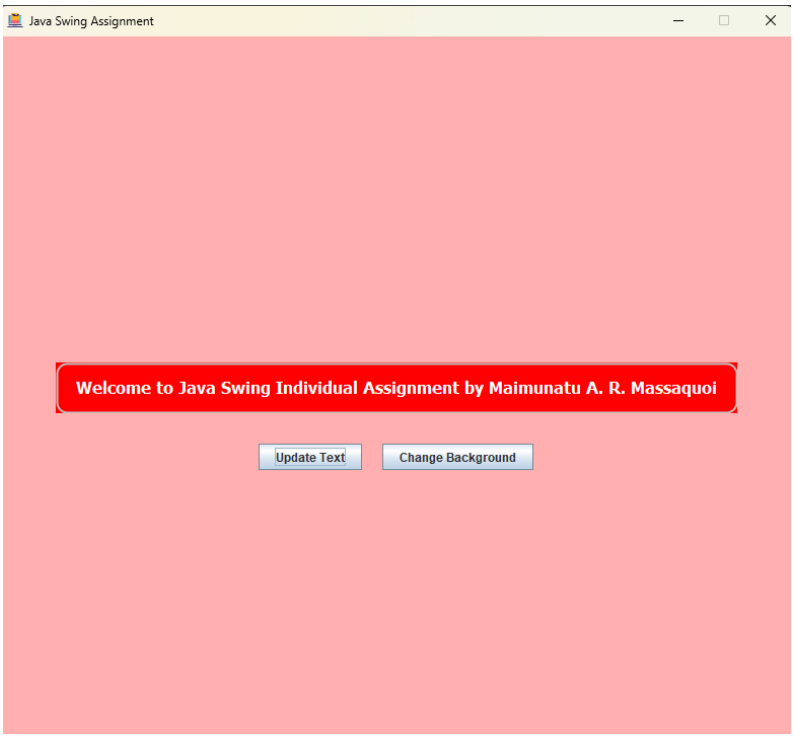
```
        public RoundedBorder(Color color, int thickness, int radius) {
            this.color = color;
            this.thickness = thickness;
            this.radius = radius;
        }

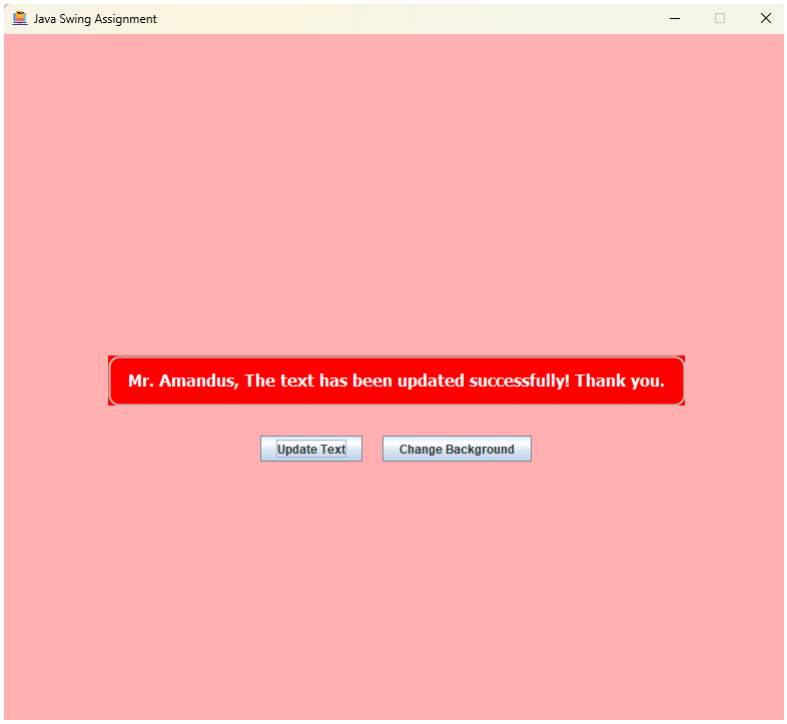
        @Override
        public void paintBorder(Component c, Graphics g, int x, int y, int width, int height) {
            Graphics2D g2d = (Graphics2D) g.create();
            // Enable Anti-Aliasing for smooth, non-pixelated curved edges
            g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
            g2d.setColor(color);
            g2d.setStroke(new BasicStroke(thickness));

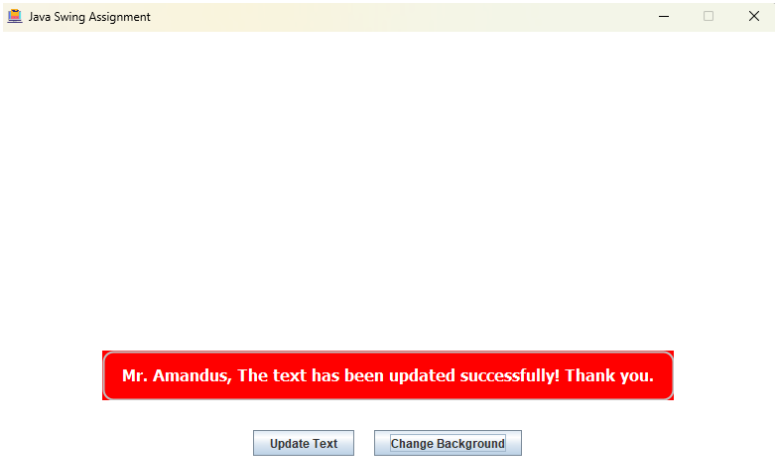
            // Draw the rounded border framework adjusted for thickness
            g2d.drawRoundRect(x + thickness / 2, y + thickness / 2,
                    width - thickness, height - thickness,
                    radius, radius);
            g2d.dispose();
        }
    }

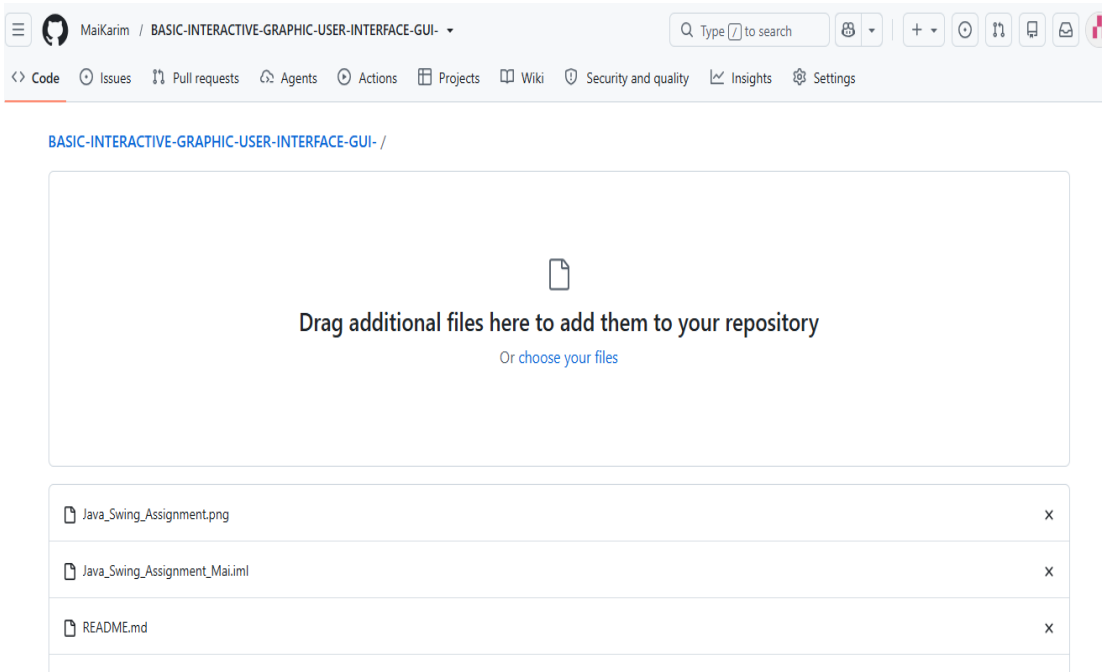
    // Main method
    public static void main(String[] args) {
        SwingUtilities.invokeLater(new Runnable() {
            @Override
            public void run() {
                new SwingAssignment();
            }
        });
    }
}
```

APPLICATION SCREENSHOTS






Java Swing Assignment
Mr. Amandus, The text has been updated successfully! Thank you.
Update Text
Change Background


MaiKarim / BASIC-INTERACTIVE-GRAPHIC-USER-INTERFACE-GUI-
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
BASIC-INTERACTIVE-GRAPHIC-USER-INTERFACE-GUI- /
Drag additional files here to add them to your repository
Or choose your files
Java_Swing_Assignment.png
Java_Swing_Assignment_Mai.iml
README.md

TESTING

Test	Action	Expected Result	Status
1	Run the program	800 x 800 Java Swing window opens	Pass
2	Click the close button	Application closes	Pass
3	Click Change Text	JLabel text changes	Pass
4	Click Change Background	JPanel background changes	Pass
5	Check application icon	Custom icon appears	Pass

CONCLUSION

In summary, this project successfully demonstrates the design and development of a foundational Java Swing graphical user interface (GUI)**.** The implementation seamlessly integrates core framework components including JFrame, JPanel, JLabel, and ImageIcon while practically applying fundamental object-oriented programming (OOP) principles such as inheritance and custom constructors. Furthermore, the inclusion of interactive buttons successfully validates the application's runtime responsiveness through dynamic text and background color modifications. Beyond software execution, the project encompasses the full development lifecycle, showcasing proficiency in structured software documentation**,** wireframe planning, and GitHub version control workflows**.**

REFERENCE

1. **Oracle Corporation. (2023). Creating a GUI with JFC/Swing*. Java Documentation***

2. **Oracle. (n.d.). Creating a GUI with Swing. Java Tutorials. Oracle.** https://docs.oracle.com/javase/tutorial/uiswing/

3. **Oracle. (n.d.). The Swing API. Java Platform, Standard Edition Documentation. Oracle.** https://docs.oracle.com/en/java/javase/

4. **Deitel, P., & Deitel, H. (2017). Java How to Program, Early Objects (11th ed.). Pearson**.

5. **Figma. (2026). Figma Design Systems and Wireframing Documentation. Figma Help Center.**

6. **JetBrains. (n.d.). IntelliJ IDEA Documentation. JetBrains.** https://www.jetbrains.com/help/idea/

7. **GitHub. (n.d.). GitHub Documentation. GitHub.** https://docs.github.com/

8. **https://chatgpt.com/**

9. **https://claude.ai/**